

WEEK 5 LEARNING GUIDE

BUILDING A PRODUCTION-READY SERVERLESS PLATFORM

1. INTRODUCTION

The previous week concluded with a fully functional serverless image processing platform capable of securely uploading images, processing them asynchronously using an event-driven architecture, and storing image metadata in Amazon DynamoDB. While the application was functionally complete, it still represented a relatively simple serverless architecture.

This week focused on understanding how cloud applications evolve as they move towards production-ready deployments. Instead of introducing major application features, the emphasis shifted towards improving the underlying architecture by exploring relational databases, private networking, caching, deployment automation, and monitoring.

The project involved replacing Amazon DynamoDB with PostgreSQL hosted on Amazon RDS, deploying backend resources inside an Amazon VPC, introducing Redis through Amazon ElastiCache, automating deployments using AWS CodePipeline and CodeBuild, securely managing configuration using Systems Manager Parameter Store, and monitoring the application using Amazon CloudWatch.

By the end of the week, I had a much better understanding of the architectural components that support production cloud applications and how these services work together to improve security, scalability, maintainability and operational reliability.

2. CORE CONCEPTS LEARNED

RELATIONAL DATABASES WITH AMAZON RDS POSTGRESQL

Until this point in the internship, every application had used Amazon DynamoDB for storing data. Since DynamoDB was the first cloud database I had worked with, it was easy to assume that NoSQL databases were the default choice for cloud applications. This week introduced PostgreSQL hosted on Amazon RDS and demonstrated that different applications require different database models depending on how their data is organised and accessed.

The biggest takeaway was that choosing a database is not about selecting the "best" database, but selecting the one that best matches the application's requirements.

What I Understood

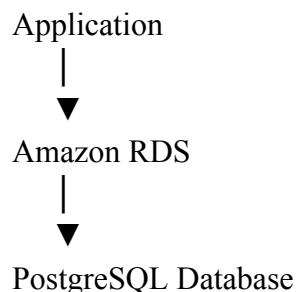
Relational databases organise information into **tables** consisting of rows and columns, similar to spreadsheets. Each table represents a particular type of data, while each row stores information about a single record.

For example, the image metadata in our project naturally fits into a relational table.

```
CREATE TABLE images (  
    image_id UUID PRIMARY KEY,  
    user_id VARCHAR(255),  
    filename VARCHAR(255),  
    status VARCHAR(20),  
    created_at TIMESTAMP  
);
```

Unlike DynamoDB, where each record is stored as an independent item, relational databases enforce a predefined structure for every record. This consistency makes querying, validation and maintaining relationships between data much easier.

Rather than installing and maintaining a PostgreSQL server ourselves, AWS provides **Amazon RDS (Relational Database Service)**, a managed service that handles database provisioning, backups, patching and maintenance while allowing developers to focus only on the application.



Using Amazon RDS meant that we could work with PostgreSQL without worrying about managing the underlying database infrastructure.

Tables and Rows

A relational database stores information inside tables.

Each table consists of:

- Columns that define the structure of the data.
- Rows that represent individual records.

For this project, every uploaded image corresponded to one row in the **images** table.

Images

image_id	filename	status
101	cat.jpg	COMPLETED
102	dog.png	PROCESSING

This structured representation makes the data easier to query, update and maintain compared to storing independent key-value items.

Primary Keys

Every table requires a way to uniquely identify each record.

This is achieved using a **Primary Key**.

In our project:

image_id UUID PRIMARY KEY

ensures that every uploaded image has a unique identifier.

Without a primary key, multiple records could represent the same image, making updates and deletions unreliable.

SQL (Structured Query Language)

Unlike DynamoDB, which is accessed using SDK operations such as **GetItem()** or **PutItem()**, relational databases are queried using SQL.

For example, retrieving an image record only requires:

```
SELECT *  
FROM images  
WHERE image_id = %s;
```

Similarly, inserting metadata becomes:

```
INSERT INTO images (  
    image_id,  
    filename,  
    status  
)  
VALUES (%s, %s, %s);
```

One thing I appreciated while working with SQL was how expressive it is. Instead of describing *how* to retrieve the data, SQL allows us to describe *what* data we need, while the database determines the most efficient way to execute the query.

Indexes

As databases grow larger, searching every row for every request becomes inefficient.

Indexes solve this problem by creating optimized lookup structures for frequently queried columns.

For example, users in our application frequently retrieved all of their uploaded images.

```
CREATE INDEX idx_user_images  
ON images (user_id) ;
```

Without an index:

Request
|
Scan Every Row
|
Find Matching Records

With an index:

Request
|
Lookup Index
|
Retrieve Matching Rows

Indexes improve query performance significantly but also introduce additional overhead whenever records are inserted or updated because the index itself must also be maintained.

Learning Through Implementation

Migrating from DynamoDB to PostgreSQL highlighted that changing the storage layer does not necessarily require redesigning the application. The API endpoints, image processing workflow and authentication flow all remained unchanged. Only the database operations responsible for storing and retrieving metadata had to be rewritten.

This demonstrated the importance of separating business logic from data storage. Since the application interacted with the database through well-defined operations, replacing DynamoDB with PostgreSQL became much simpler than rebuilding the application from scratch.

Another important lesson was that relational and NoSQL databases are designed to solve different problems. Understanding their strengths and limitations is more valuable than trying to identify one as universally better than the other.

Application in this Project

Amazon RDS PostgreSQL became the primary metadata store for the application.

The migration involved:

- Creating the database schema using a dedicated Migration Lambda.
- Replacing DynamoDB operations with SQL queries.
- Storing image metadata inside relational tables.
- Updating the Processor Lambda to modify PostgreSQL records after image processing.
- Using indexed queries to efficiently retrieve images belonging to a particular user.

Although the application's external behaviour remained the same, replacing DynamoDB with PostgreSQL provided valuable insight into how production applications use relational databases to manage structured data.

AMAZON VIRTUAL PRIVATE CLOUD (AMAZON VPC)

Until this point in the internship, most AWS services interacted with each other without requiring any knowledge of networking. Services such as Amazon S3, Lambda, DynamoDB and API Gateway are managed services that are accessible over the AWS network without explicitly creating a private network. This week introduced **Amazon Virtual Private Cloud (VPC)**, which demonstrated that many production services, especially databases and caches, are deployed inside isolated private networks rather than being publicly accessible.

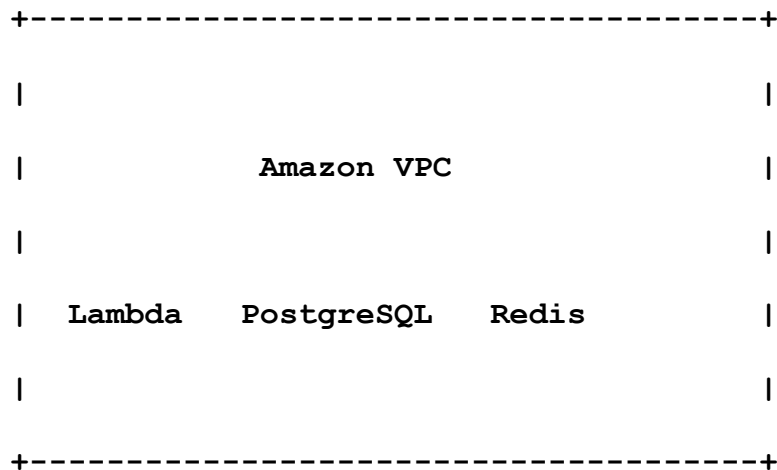
One of the biggest takeaways was understanding that **security begins with network design**. Instead of allowing every service to communicate over the internet, a VPC creates a private environment where only authorised resources can communicate with one another.

What is an Amazon VPC?

An Amazon VPC is a logically isolated network within AWS where AWS resources can be deployed securely. It gives complete control over how resources communicate by allowing us to define IP ranges, subnets, routing rules and firewall policies.

Instead of deploying every resource directly onto the internet, services can be placed inside a private network and exposed only when necessary.

AWS Cloud



In our project, both PostgreSQL and Redis were deployed inside the VPC, ensuring they were not directly accessible from the internet.

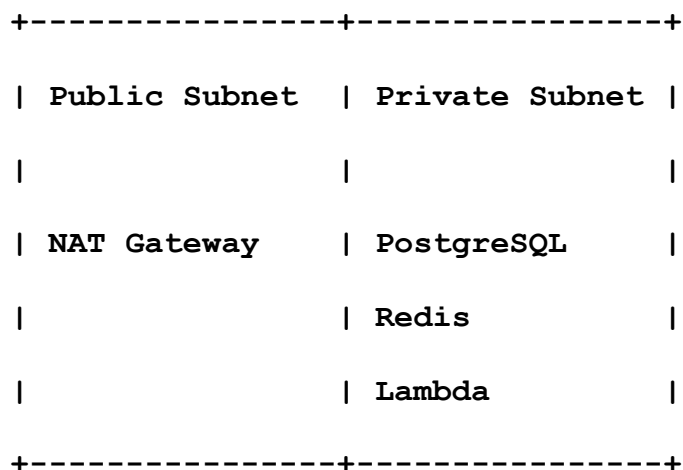
Public and Private Subnets

A VPC is divided into smaller networks called **subnets**.

There are two common types:

- **Public Subnet** – Resources can communicate directly with the internet.
- **Private Subnet** – Resources cannot be accessed directly from the internet.

Amazon VPC



Sensitive resources such as databases should almost always be deployed in private subnets. This reduces the attack surface by preventing direct external access.

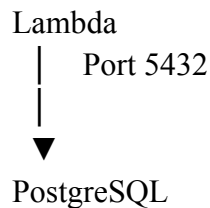
Security Groups

A **Security Group** acts as a virtual firewall for AWS resources.

Instead of allowing every service to communicate freely, Security Groups define:

- Which resources can initiate connections.
- Which ports are accessible.
- Which traffic should be blocked.

For example, PostgreSQL communicates using port **5432**.



Only the Lambda function was permitted to connect to the database. Any other incoming traffic was denied.

This follows the **Principle of Least Privilege**, where resources receive only the permissions they require.

DB Subnet Groups

Unlike services such as Lambda, Amazon RDS requires multiple subnets for deployment.

A **DB Subnet Group** specifies the private subnets where the database is allowed to run.

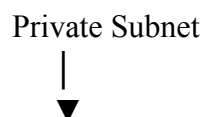
Using multiple subnets improves availability because AWS can deploy database instances across different Availability Zones if required.

NAT Gateway

Deploying Lambda inside a private subnet introduces an interesting challenge.

Although the Lambda function can now securely access PostgreSQL and Redis, it loses direct internet access.

Whenever a private resource needs outbound internet connectivity—for example, downloading Python dependencies or communicating with AWS services—a **NAT Gateway** is used.



NAT Gateway



Internet

The NAT Gateway allows resources inside the private subnet to initiate outbound connections while preventing unsolicited inbound traffic.

Example Configuration

The following Terraform configuration places a Lambda function inside a VPC.

```
vpc_config {  
  subnet_ids      = var.private_subnet_ids  
  security_group_ids = [aws_security_group.lambda.id]  
}
```

Instead of running on AWS's default network, the Lambda function now executes inside the specified private subnets and communicates with other resources using the VPC network.

Learning Through Implementation

Before this week, networking felt like a separate topic that mainly concerned system administrators. Working with Amazon VPC showed that networking decisions directly influence how cloud applications are designed and secured.

Initially, it seemed unnecessary to place Lambda inside a VPC. However, once PostgreSQL and Redis were deployed within private subnets, it became clear that the application itself also needed private network access. This demonstrated that infrastructure decisions often affect multiple components of the architecture rather than individual services in isolation.

Application in this Project

The Week 4 application already had a functioning event-driven workflow. Week 5 retained the same architecture while improving its security.

- Amazon RDS PostgreSQL was deployed inside private subnets.
- Amazon ElastiCache Redis was also deployed inside the VPC.
- The backend Lambda functions were configured to run inside the same VPC.
- Security Groups restricted communication so that only authorised resources could access the database and cache.
- A NAT Gateway enabled outbound internet access for private resources whenever required.

Although these networking changes were invisible to end users, they significantly improved the security of the overall architecture and introduced one of the most important concepts encountered during the internship so far.

AMAZON ELASTICACHE (REDIS)

As the application evolved, another important question emerged: **Should every request always query the database?**

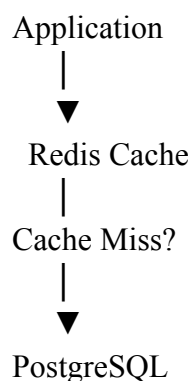
For a small application, directly reading from the database for every request may not cause noticeable issues. However, as the number of users and requests increases, repeatedly executing the same queries can place unnecessary load on the database and increase response times. This week introduced **Amazon ElastiCache** and the concept of **caching** as a solution to this problem.

Rather than replacing the database, Redis works alongside it by temporarily storing frequently accessed data in memory, allowing future requests to be served much faster.

What is Amazon ElastiCache?

Amazon ElastiCache is a fully managed in-memory data store that supports caching technologies such as **Redis** and **Memcached**.

Unlike PostgreSQL, which stores data permanently on disk, Redis stores data in **memory (RAM)**. Since memory access is significantly faster than disk access, retrieving cached data is almost instantaneous.

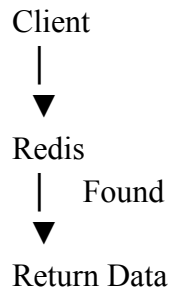


Redis should not be viewed as a replacement for a database. Instead, it acts as a temporary storage layer that sits in front of the database and reduces the number of repeated queries.

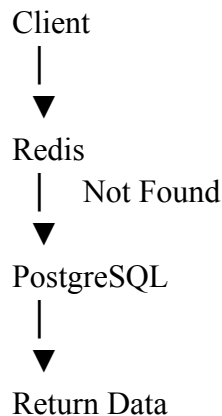
Cache Hits and Cache Misses

Whenever the application needs data, it first checks whether the information already exists in Redis.

If the data exists, it is called a **Cache Hit**.



If the data is not present, it is called a **Cache Miss**.



After retrieving the data from PostgreSQL, the application usually stores a copy inside Redis so that future requests can be served directly from the cache.

Cache-Aside Pattern

The most common caching strategy, and the one implemented in this project, is the **Cache-Aside Pattern**.

The workflow is simple:

1. Check Redis for the requested data.
2. If found, return the cached data.
3. Otherwise, query PostgreSQL.
4. Store the retrieved data in Redis.
5. Return the response to the client.

A simplified implementation looks like this:

```
cached = redis.get(cache_key)
if cached:
    return cached
```

```
data = get_image_from_database(image_id)
redis.set(cache_key, data)
return data
```

This pattern keeps Redis independent from the database while ensuring that frequently requested data becomes progressively faster to retrieve.

Time-To-Live (TTL)

One limitation of caching is that cached data can become outdated if the underlying database changes.

Redis addresses this using a **Time-To-Live (TTL)**.

When storing data, an expiration time can also be specified.

```
redis.set(
    cache_key,
    image_data,
    ex=300
)
```

In this example, the cached entry automatically expires after **300 seconds**, ensuring that stale data does not remain in memory indefinitely.

TTL helps balance performance with data freshness without requiring manual cleanup.

Cache Invalidation

Another important concept introduced this week was **cache invalidation**.

Whenever cached data is modified or deleted in the database, the corresponding cache entry must also be removed. Otherwise, users may continue receiving outdated information.

For example:

```
redis.delete(cache_key)
```

Although deleting a cache entry is straightforward, deciding **when** it should be invalidated is often one of the more challenging aspects of designing a caching strategy.

Learning Through Implementation

Initially, it seemed reasonable to cache every API response in order to maximise performance. However, while implementing Redis, it became clear that caching also

introduces additional complexity. Every cached response must eventually be refreshed or invalidated whenever the underlying data changes.

This highlighted an important engineering principle: **caching should only be introduced when it provides a measurable benefit**. Unnecessary caching increases system complexity without significantly improving performance.

Application in this Project

Redis was introduced to reduce repeated database queries for frequently accessed image metadata.

During development, the original design proposed caching both the image listing endpoint (**GET /images**) and the single image retrieval endpoint (**GET /images/{imageId}**).

After evaluating the implementation, this approach was simplified.

- ***Original approach***
 - Cache both image listing and individual image retrieval endpoints.
- ***Problem discovered***
 - The image listing endpoint changed frequently whenever users uploaded or deleted images, making cache invalidation more complicated than the performance benefit justified.
- ***Final implementation***
 - Caching was removed from **GET /images**.
 - Only **GET /images/{imageId}** continued to use Redis because individual image metadata changed much less frequently.

This decision demonstrated that caching should be applied selectively rather than universally. Understanding **where not to use a cache** was just as valuable as learning how to implement one.

CONTINUOUS INTEGRATION AND CONTINUOUS DEPLOYMENT (CI/CD)

Until this point, every infrastructure change involved manually running Terraform commands and manually packaging Lambda functions before deployment. While this approach worked during development, it quickly became apparent that it would be difficult to maintain as the project grew. Forgetting a deployment step or accidentally using an outdated package could easily introduce inconsistencies between the local environment and AWS.

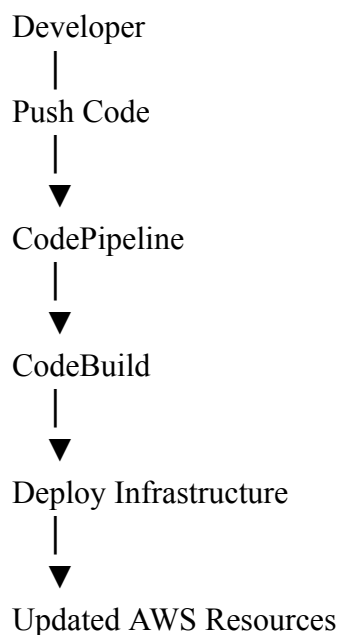
This week introduced **Continuous Integration and Continuous Deployment (CI/CD)**, allowing the entire deployment process to be automated. Rather than deploying changes

manually, AWS services were responsible for building, validating and deploying the latest version of the application whenever new code was committed.

What is CI/CD?

CI/CD is a software engineering practice that automates the process of integrating code changes and deploying applications.

Instead of manually executing every deployment step, a CI/CD pipeline performs them automatically in a predefined sequence.



Automating deployments improves consistency, reduces manual errors and ensures that every deployment follows exactly the same process.

AWS CodePipeline

AWS CodePipeline is a managed service that orchestrates the entire deployment workflow.

A pipeline consists of multiple stages, each responsible for a specific task.

Typical stages include:

- **Source** – Retrieves the latest project files from the repository.
- **Build** – Compiles, packages and prepares the application.
- **Deploy** – Updates AWS infrastructure and application resources.

Rather than executing these steps manually, CodePipeline coordinates them automatically whenever the source code changes.

AWS CodeBuild

While CodePipeline manages the workflow, **AWS CodeBuild** performs the actual build process.

During a build, CodeBuild runs inside a temporary build environment where it executes commands defined inside a **buildspec.yml** file.

For example:

```
version: 0.2
phases:
  build:
    commands:
      - terraform init
      - terraform apply -auto-approve
```

This configuration demonstrates an important idea: instead of remembering deployment commands, they become part of the project's source code. Every deployment therefore follows the same repeatable process.

buildspec.yml

The **buildspec.yml** file defines exactly what CodeBuild should execute.

A simplified structure is shown below.

```
buildspec.yml
├─ install
├─ pre_build
├─ build
└─ post_build
```

Each phase serves a different purpose.

- **Install** – Prepare the build environment.
- **Pre-build** – Download dependencies or initialise tools.
- **Build** – Compile code and deploy infrastructure.
- **Post-build** – Perform cleanup or generate deployment artifacts.

Separating the workflow into phases makes complex deployments easier to organise and maintain.

Learning Through Implementation

One of the biggest changes this week was shifting from thinking of deployment as a manual task to treating it as part of the application's architecture.

Initially, running Terraform locally felt sufficient because the project was being developed by a single person. However, introducing CodePipeline and CodeBuild demonstrated how automation ensures that deployments remain consistent regardless of who performs them or where they are executed.

Another important takeaway was that infrastructure should be version controlled just like application code. Since Terraform configuration already described the infrastructure, automating its deployment through CI/CD completed the Infrastructure as Code workflow introduced during Week 1.

Application in this Project

The final implementation automated the deployment of both the application and its supporting infrastructure.

The deployment pipeline performed tasks such as:

- Retrieving the latest source code.
- Packaging Lambda functions.
- Executing Terraform to provision or update AWS resources.
- Applying database schema migrations.
- Deploying the latest frontend assets.

Although the deployment process became significantly more sophisticated, the application's functionality remained unchanged. The primary improvement was ensuring that every deployment followed the same repeatable workflow without requiring manual intervention.

AWS SYSTEMS MANAGER PARAMETER STORE

As the number of AWS resources increased, another practical problem became apparent. Services such as Lambda functions required database endpoints, Redis endpoints and other configuration values in order to operate correctly.

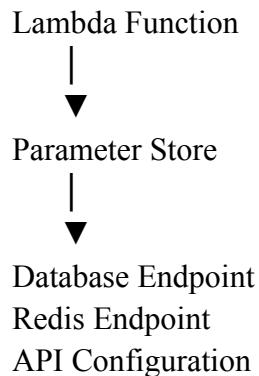
Initially, it might seem convenient to hardcode these values directly inside the application. However, this quickly becomes difficult to maintain whenever infrastructure changes. Updating configuration would require modifying the application code and redeploying the entire service.

This week introduced **AWS Systems Manager Parameter Store**, which provides a central location for storing configuration values separately from the application.

What is Parameter Store?

Parameter Store is a managed configuration service that allows applications to retrieve configuration values at runtime.

Instead of embedding configuration directly into the code, applications request the required parameters whenever they start.



Separating configuration from application logic makes the system easier to maintain while reducing the likelihood of configuration errors.

Centralised Configuration

Rather than storing configuration across multiple Lambda functions, Parameter Store keeps everything in a single location.

Typical parameters include:

- Database hostname.
- Database port.
- Redis endpoint.
- Application configuration values.

This approach ensures that configuration changes can be managed independently of the application code.

Runtime Retrieval

Instead of hardcoding values, Lambda retrieves them when required.

A simplified example is shown below.

```
ssm = boto3.client("ssm")
parameter = ssm.get_parameter(
    Name="/image-api/database/host"
)
```

The application no longer needs to know where the database is hosted. It simply requests the appropriate parameter from Systems Manager.

Learning Through Implementation

Previously, configuration values were often treated as part of the application itself. Working with Parameter Store demonstrated that configuration is another component of the infrastructure and should be managed independently.

Separating configuration from code also makes it easier to deploy the same application across different environments, such as development, testing and production, without modifying the application logic.

Application in this Project

Parameter Store was used to provide backend services with infrastructure-specific configuration such as database and Redis connection details.

This removed the need to hardcode resource identifiers inside the Lambda functions and ensured that configuration could be updated without modifying the application code itself.

AMAZON CLOUDWATCH

Until this point, verifying whether the application was working correctly largely involved manually testing API endpoints or inspecting Lambda logs whenever an error occurred. While this approach was sufficient during development, it would quickly become impractical as the application grew. A production application requires a way to continuously observe the health of its services and notify developers whenever something goes wrong.

This week introduced **Amazon CloudWatch**, AWS's monitoring and observability service. Instead of waiting for users to report failures, CloudWatch continuously collects operational data from AWS resources, making it possible to detect problems much earlier.

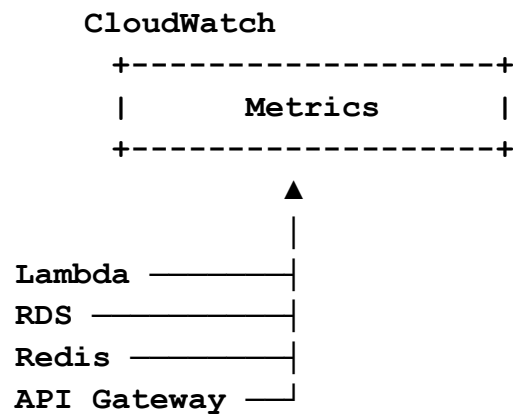
What is Amazon CloudWatch?

Amazon CloudWatch is a managed monitoring service that collects information about AWS resources and applications.

This information generally falls into three categories:

- **Logs** – Detailed records generated by applications.
- **Metrics** – Numerical measurements collected over time.
- **Alarms** – Notifications triggered when metrics exceed predefined thresholds.

Together, these components provide visibility into how an application behaves in production.



Rather than monitoring individual services separately, CloudWatch acts as a central location for collecting operational information across the entire application.

CloudWatch Logs

Every Lambda execution produces logs that can be viewed through CloudWatch.

These logs are extremely useful for:

- Debugging runtime errors.
- Verifying application behaviour.
- Recording important events.
- Tracking request execution.

For example, logging useful information inside a Lambda function is as simple as:

```
import logging
logger = logging.getLogger()
logger.info("Image processed successfully.")
```

Instead of manually printing information to the console, structured logs become available inside CloudWatch for later inspection.

CloudWatch Metrics

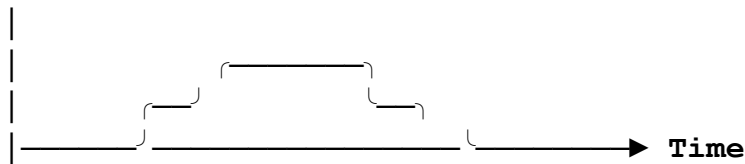
While logs describe **what happened**, metrics describe **how the application is performing**.

Examples include:

- Lambda Invocations
- Lambda Errors
- Function Duration
- Database CPU Utilisation
- Redis Memory Usage

These measurements are continuously collected and plotted over time, making it easy to identify performance trends or unexpected behaviour.

Lambda Invocations



Unlike logs, metrics provide a high-level overview of system health rather than individual execution details.

CloudWatch Alarms

Metrics become even more useful when combined with **CloudWatch Alarms**.

An alarm continuously monitors a metric and performs an action whenever a specified threshold is exceeded.

For example:

Lambda Errors > 5



CloudWatch Alarm



Notification

Rather than requiring developers to constantly monitor dashboards, alarms automatically notify them whenever abnormal behaviour is detected.

This allows issues to be identified and investigated much more quickly.

Learning Through Implementation

One important lesson this week was recognising the difference between **debugging** and **monitoring**.

During development, debugging usually begins *after* a problem has already occurred. Monitoring, however, is proactive. By continuously collecting logs and metrics, CloudWatch makes it possible to identify failures, performance degradation and unusual behaviour before they significantly affect users.

Another key takeaway was that observability is an essential part of application design. Building the application is only one part of the process; understanding how it behaves after deployment is equally important.

Application in this Project

CloudWatch was integrated to improve the observability of the image processing application.

It was used to:

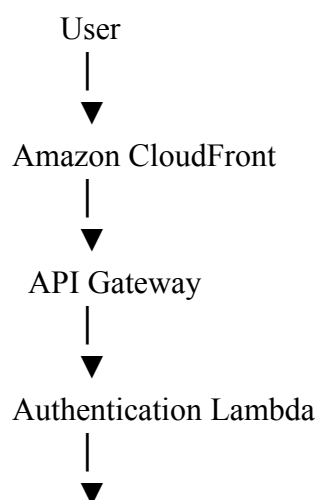
- Collect execution logs from Lambda functions.
- Monitor the health of backend services.
- Track important operational metrics.
- Configure alarms for critical application events.

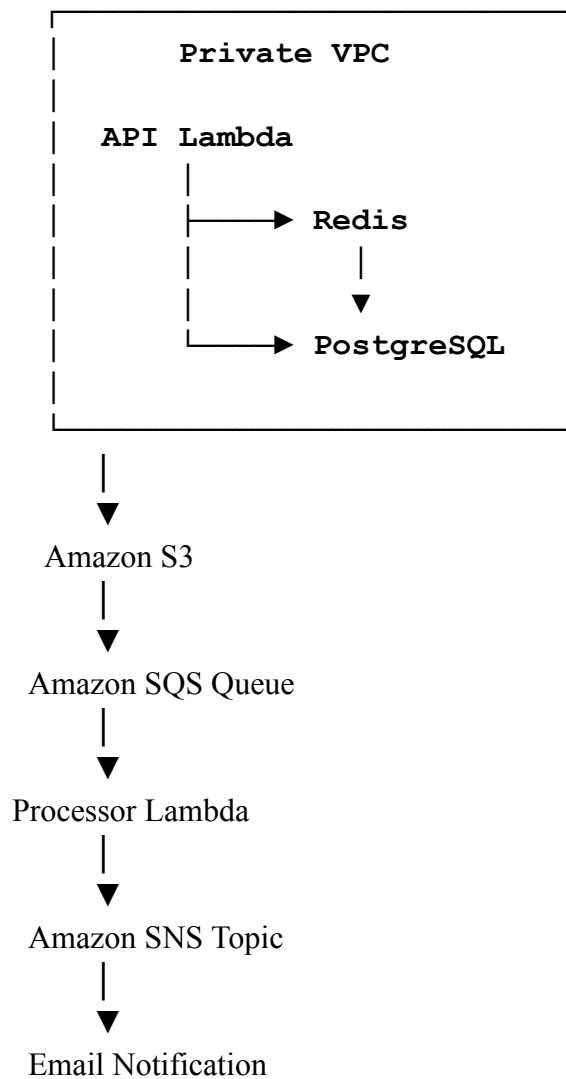
Although CloudWatch introduced no new application functionality, it significantly improved the ability to monitor, troubleshoot and maintain the system after deployment.

3. PROJECT IMPLEMENTATION

By the end of Week 5, the image processing platform had evolved from a fully serverless application into a more production-oriented cloud architecture. While the overall workflow remained similar to Week 4, several infrastructure components were redesigned to improve scalability, security and maintainability.

The final architecture is shown below.





Unlike previous weeks, the primary objective was not to introduce new user-facing features but to strengthen the application's infrastructure. Existing workflows, such as image uploads, asynchronous processing and notifications, remained unchanged while the underlying architecture became more production ready.

MIGRATING FROM DYNAMODB TO POSTGRESQL

The first architectural change involved replacing DynamoDB with PostgreSQL.

Original approach

Image metadata was stored in Amazon DynamoDB.

Reason for change

This week's objective was to understand relational databases and SQL while working with a production-style managed database.

Final implementation

PostgreSQL hosted on Amazon RDS became the primary metadata store. Existing DynamoDB operations were replaced with SQL queries while preserving the application's external behaviour.

Learning outcome

Separating business logic from the storage layer makes it possible to migrate between different database technologies without redesigning the entire application.

SECURING BACKEND RESOURCES USING AMAZON VPC

Introducing PostgreSQL and Redis required both services to be deployed inside a private network.

Instead of exposing backend resources publicly, the architecture was redesigned so that:

- PostgreSQL resided inside private subnets.
- Redis was deployed alongside the database.
- Backend Lambda functions were configured to execute within the same VPC.
- Security Groups restricted communication to authorised services only.

Although this increased the networking complexity, it significantly improved the overall security of the application.

INTRODUCING REDIS CACHING

Redis was added to reduce unnecessary database queries.

Original approach

Cache both image listing and individual image retrieval endpoints.

Problem discovered

Frequently changing image collections required constant cache invalidation, making the cache difficult to maintain.

Final implementation

Caching was retained only for individual image lookups, while the image listing endpoint continued to query PostgreSQL directly.

Learning outcome

Caching should be introduced selectively after evaluating both its performance benefits and maintenance overhead.

AUTOMATING DEPLOYMENTS WITH CI/CD

As the project became larger, manually packaging Lambda functions, running Terraform commands and updating AWS resources became increasingly repetitive.

Instead of relying on manual deployments, a CI/CD pipeline was introduced.

Original approach

Package Lambda functions manually.

↓

Run Terraform commands locally.

↓

Verify deployment manually.

Problem discovered

The deployment process became time-consuming and prone to human error. Missing even a single step could leave the deployed infrastructure inconsistent with the local codebase.

Final implementation

AWS CodePipeline and CodeBuild automated the deployment workflow. Whenever new changes were pushed to the repository, the pipeline automatically:

- Retrieved the latest source code.
- Built the application.
- Packaged the Lambda functions.
- Executed Terraform to provision or update infrastructure.
- Applied database migrations before deploying the latest application version.

Developer

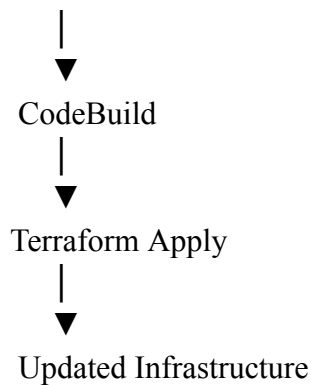
|

Push Changes

|



CodePipeline



Learning outcome

Deployment is an important part of software engineering, not just an operational task. Automating deployments improves consistency, reduces manual effort and ensures that infrastructure and application code remain synchronized.

MANAGING CONFIGURATION WITH SYSTEMS MANAGER PARAMETER STORE

As new services such as PostgreSQL and Redis were introduced, the application required additional configuration values including database endpoints, ports and connection details.

Initially, storing these values directly inside the Lambda functions seemed like the simplest approach.

Original approach

Hardcode infrastructure-specific configuration inside the application.

Problem discovered

Whenever infrastructure changed, the application code also needed to be modified and redeployed, making maintenance more difficult.

Final implementation

Infrastructure configuration was moved into AWS Systems Manager Parameter Store. During execution, Lambda functions retrieved the required parameters dynamically.

```
ssm = boto3.client("ssm")
db_host = ssm.get_parameter(
    Name="/image-api/database/host"
) ["Parameter"] ["Value"]
```


This separated application logic from infrastructure configuration.

Learning outcome

Application code should remain independent of deployment-specific configuration. Storing configuration externally makes applications easier to maintain and deploy across different environments.

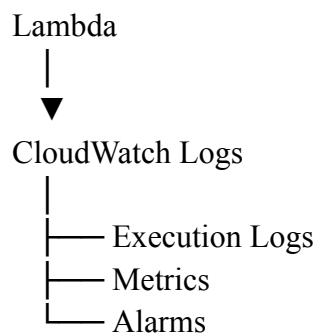
MONITORING THE APPLICATION WITH CLOUDWATCH

Deploying an application is only the first step; understanding how it behaves after deployment is equally important.

To improve operational visibility, Amazon CloudWatch was integrated into the architecture.

CloudWatch was configured to:

- Collect execution logs from Lambda functions.
- Monitor application performance metrics.
- Generate alarms for abnormal application behaviour.



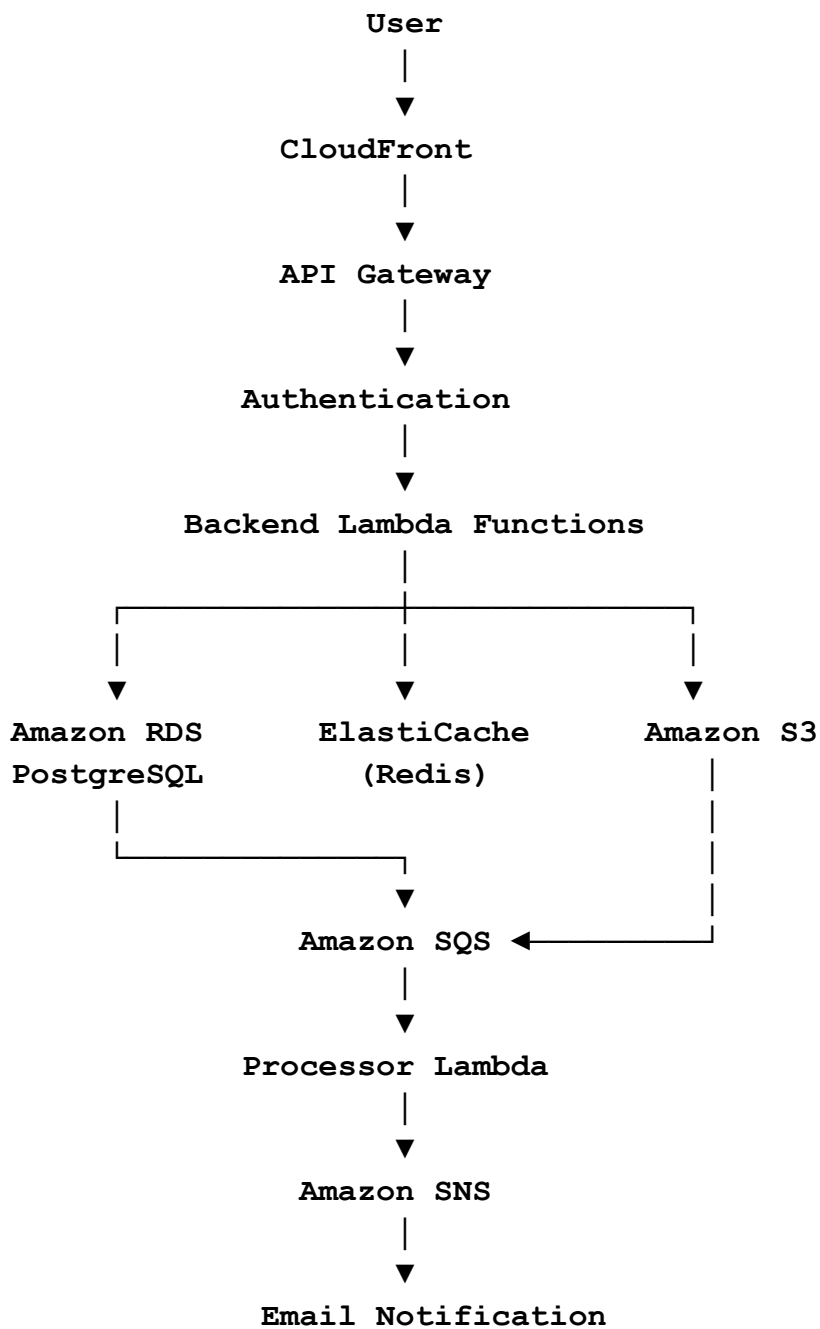
Rather than manually inspecting every service, CloudWatch provided a centralized view of the application's health.

Learning outcome

Monitoring should be treated as part of the application's architecture rather than something added only after deployment. Good observability makes debugging faster and helps identify problems before they impact users.

FINAL ARCHITECTURE

The completed application combined everything learned over the previous five weeks into a single cloud-native architecture.



Compared to the previous week's architecture, the application's business logic remained largely unchanged. The improvements focused on strengthening the underlying infrastructure through:

- A relational database for structured metadata storage.
- Private networking using Amazon VPC.
- Redis caching for improved read performance.
- Automated deployments through CI/CD.
- Centralized configuration management.
- Application monitoring and observability.

These changes transformed the application from a functional serverless prototype into a production-oriented cloud architecture while preserving the event-driven design established during the earlier weeks.

4. KEY LEARNINGS & TAKEAWAYS

This week shifted the focus from building application features to understanding the infrastructure that supports production cloud systems. Some of the most important takeaways were:

- Relational databases and NoSQL databases solve different problems, and choosing the appropriate database depends on the application's data model and access patterns.
- Networking plays a fundamental role in cloud security. Deploying services inside private VPCs significantly reduces the application's attack surface.
- Caching should be introduced selectively. While Redis can greatly improve performance, poorly chosen caching strategies can introduce unnecessary complexity.
- Infrastructure as Code becomes significantly more valuable when combined with automated CI/CD pipelines, ensuring consistent and repeatable deployments.
- Application configuration should be managed independently from application code using services such as Systems Manager Parameter Store.
- Monitoring and observability are essential components of production systems. CloudWatch provides the visibility required to diagnose failures and monitor application health after deployment.

By the end of Week 5, the image processing platform had evolved from a serverless application into a production-style cloud system incorporating secure networking, relational databases, caching, automated deployments and operational monitoring. More importantly, this week reinforced that designing cloud applications involves much more than implementing business logic—architectural decisions surrounding infrastructure, deployment and observability are equally critical to building reliable and maintainable systems.